

POS TECH

ARQUITETURA E DESENVOLVIMENTO EM JAVA

FASE 3 | AULA 01 -

INTRODUÇÃO AO GRPC

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

SUMÁRIO

O QUE VEM POR AÍ? 3

HANDS ON 4

SAIBA MAIS 5

O QUE VOCÊ VIU NESTA AULA? 14

REFERÊNCIAS 15

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

O QUE VEM POR AÍ?

Você já parou para pensar em como sistemas espalhados pelo mundo conseguem se comunicar de forma eficiente, mesmo usando diferentes linguagens de programação e infraestruturas? É nesse contexto que o gRPC se destaca. Vamos explorar como as chamadas de procedimento remoto (RPC) evoluíram para atender às demandas atuais de desempenho e escalabilidade, apresentando o gRPC como uma tecnologia que une simplicidade e eficiência.

Nesta aula, você aprenderá o que é o gRPC, suas vantagens e como ele se conecta ao ProtoBuf, um mecanismo agnóstico a linguagens de programação criada para a serialização de dados. Vamos entender os tipos de dados e mensagens no ProtoBuf, criar nossos próprios serviços e explorar os quatro tipos de comunicação que o gRPC suporta.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

HANDS ON

Nesta aula, começaremos entendendo os conceitos que sustentam o gRPC. Primeiramente, exploraremos o que é RPC (Remote Procedure Call) e como essa abordagem facilita a comunicação entre sistemas distribuídos. Em seguida, discutiremos o que é o gRPC, suas principais vantagens e como ele utiliza o ProtoBuf como formato de serialização de dados eficiente e compacto. Durante essa etapa, também abordaremos o que é o protoc, o compilador que transforma as definições do ProtoBuf em código utilizável nas aplicações.

Avançando, exploraremos os quatro modelos de comunicação que o gRPC suporta: Unary, Server-streaming, Client-streaming e Bidirectional-streaming. Compreenderemos como esses modelos permitem diferentes fluxos de interação entre cliente e servidor, ajustando-se às necessidades de comunicação das aplicações modernas.

Por fim, colocaremos todos esses conceitos em prática ao criar uma demo. Durante essa atividade, definiremos um arquivo ProtoBuf para especificar as mensagens, utilizaremos o protoc para gerar o código necessário e implementaremos um serviço funcional utilizando gRPC. O código utilizado nessa aula está disponível no GitHub.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

SAIBA MAIS

O RPC, ou Remote Procedure Call, é um protocolo essencial quando falamos de comunicação entre sistemas, especialmente em ambientes distribuídos. Ele permite que um sistema execute funções ou procedimentos localizados em outro sistema remoto como se estivessem na mesma máquina. Essa abstração traz uma enorme simplificação para a pessoa desenvolvedora, que pode programar como se estivesse lidando com chamadas locais, enquanto o protocolo cuida da complexidade de traduzir essas chamadas para o contexto remoto.

Imagine dois computadores conectados por uma rede. Um deles, o cliente, faz uma solicitação de execução de uma função, enquanto o outro, o servidor, processa essa solicitação e retorna o resultado. Essa troca de informações é intermediada pelo RPC, que traduz a chamada do cliente para um formato compreensível pelo servidor e vice-versa, para que a resposta volte ao cliente de forma transparente.

O servidor RPC, peça-chave no processo, é o responsável por receber as solicitações, executar o código correspondente e devolver os resultados. Ele pode estar localizado tanto em um servidor na nuvem quanto em uma máquina local e viabiliza o uso de recursos remotos com facilidade. Essa infraestrutura é amplamente usada em serviços modernos, como aplicações na nuvem, onde a interface do usuário pode rodar localmente, mas os dados são processados em servidores remotos.

O uso do RPC traz diversas vantagens. Ele simplifica o desenvolvimento, pois abstrai a complexidade da comunicação em rede; é transparente, permitindo que pessoas desenvolvedoras trabalhem como se as chamadas fossem locais; oferece escalabilidade, sendo uma solução ideal para sistemas na nuvem e melhora a eficiência ao integrar sistemas diferentes. Contudo, como qualquer tecnologia, também apresenta desafios. A latência de rede pode ser um problema, especialmente quando os sistemas estão distantes geograficamente. Além disso, falhas na rede podem comprometer a comunicação e a confiabilidade e a depuração em sistemas distribuídos pode se tornar complexa.

A aplicação do RPC é ampla. Ele é utilizado em serviços de nuvem, onde funções remotas processam grandes volumes de dados; em sistemas distribuídos,

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

para garantir a troca eficiente de informações; e em APIs de integração, que dependem dessa comunicação para conectar plataformas distintas.

É aí que entra o gRPC. Ele é uma estrutura de código aberto, mantida pela Cloud Native Computing Foundation (CNCF), que se destaca como uma solução moderna e eficiente para comunicação entre sistemas distribuídos. Desenvolvido inicialmente pelo Google, o gRPC utiliza o protocolo HTTP/2 para proporcionar comunicação altamente performática, permitindo o streaming full-duplex, onde cliente e servidor podem trocar informações simultaneamente em tempo real. Essa característica o torna especialmente útil para arquiteturas de microsserviços e aplicações que demandam interações rápidas e contínuas.

Uma das grandes inovações do gRPC é sua abordagem para melhorar o desempenho da troca de dados. Ele utiliza um formato de comunicação binário, baseado nos proto buffers (protobufs), que garante compactação eficiente dos dados e maior velocidade na transmissão. Graças a isso, o gRPC pode ser até sete vezes mais rápido nas respostas e dez vezes mais rápido no envio de dados, quando comparado a protocolos baseados em texto, como JSON. Essa eficiência é amplificada pelas vantagens do HTTP/2, que oferece recursos como multiplexação de streams, compressão de cabeçalhos e baixa latência, características ideais para sistemas que precisam lidar com grandes volumes de informações de forma ágil.

Outra característica essencial do gRPC é sua facilidade de uso. Ele automatiza a geração de interfaces de código a partir de um arquivo de contrato, onde estão definidos os serviços e os objetos de transferência de dados (DTOs). Essa abordagem permite que pessoas desenvolvedoras se concentrem exclusivamente na lógica do sistema, enquanto o gRPC gerencia os detalhes técnicos da comunicação entre cliente e servidor. É como se você pudesse declarar funções no back-end e acessá-las diretamente no front-end, sem complicações.

No cenário de microsserviços, o gRPC oferece vantagens importantes. Ele estabelece contratos claros entre cliente e servidor, facilitando a integração e evitando ambiguidades. Além disso, recursos nativos do HTTP/2, como suporte a streaming de dados, balanceamento de carga e monitoramento, tornam a gestão dos serviços mais eficiente e robusta. Esses benefícios são especialmente relevantes em arquiteturas distribuídas, onde a desempenho e a confiabilidade são necessárias para manter a operação.

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

Mantido pela CNCF, que também é responsável por projetos como Kubernetes e Prometheus, o gRPC faz parte do ecossistema de tecnologias nativas da nuvem. Mas, para criar nossos serviços utilizando o gRPC, precisamos entender bem o protobuf. Criados pelo Google, eles são o mecanismo padrão utilizado pelo gRPC para serializar dados estruturados, proporcionando uma comunicação eficiente e compacta entre sistemas. Embora o gRPC possa ser usado com outros formatos, como JSON, o Protobuf é amplamente preferido devido ao seu desempenho superior e à forma enxuta como representa os dados.

Para começar a trabalhar com Protobuf, é necessário criar um arquivo de definição com a extensão **.proto**. Nele, você descreve as mensagens que representam os dados que deseja transmitir. Cada mensagem é composta por campos organizados como pares nome-valor. Vamos imaginar que você esteja desenvolvendo um sistema para gerenciar veículos. Um exemplo de mensagem seria:

```
message Veiculo {
  string placa = 1;
  string modelo = 2;
  int32 ano = 3;
  bool em_estoque = 4;
}
```

Código-fonte 1 – Exemplo de arquivo .proto
Fonte: elaborado pelo autor (2024)

Essa estrutura descreve um veículo com os campos placa (texto), modelo (texto), ano (número inteiro) e em_estoque (valor booleano). Esse arquivo age como um contrato claro e compartilhado entre os sistemas, garantindo que cliente e servidor entendam os dados da mesma forma.

Após definir o arquivo .**proto**, você utiliza o compilador **protoc** para gerar classes em sua linguagem de programação preferida. Essas classes possuem métodos para manipular os dados definidos no contrato. Por exemplo, se você estiver usando Java, o compilador gerará uma classe chamada Veiculo, que terá métodos como getPlaca() e setPlaca() para acessar ou definir o valor do campo placa. Além disso, a classe inclui métodos para serializar e desserializar os dados em um formato binário compacto, pronto para ser enviado pela rede.

No caso do gRPC, além de definir mensagens, você pode especificar serviços e métodos RPC diretamente no arquivo .proto. Por exemplo, em um sistema de

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

locadora de veículos, poderíamos definir um serviço chamado GestorDeVeiculos com um método CadastrarVeiculo para registrar um novo veículo:

```
     // Definição do serviço de gestão de veículos
     service GestorDeVeiculos {
       rpc     CadastrarVeiculo     (VeiculoRequest)     returns
(VeiculoResponse);
     }

     // Mensagem para a requisição de cadastro de um veículo
     message VeiculoRequest {
       string placa = 1;
       string modelo = 2;
       int32 ano = 3;
       bool em_estoque = 4;
     }

     // Mensagem para a resposta do cadastro de um veículo
     message VeiculoResponse {
       string mensagem = 1;
     }
```

Código-fonte 2 – Exemplo de arquivo .proto
Fonte: elaborado pelo autor (2024)

Com essa definição, o compilador protoc e o plugin do gRPC geram o código necessário para o cliente e o servidor. O cliente pode simplesmente chamar o método CadastrarVeiculo como se fosse uma função local, enquanto o servidor implementa a lógica para processar a requisição.

Atualmente, a versão mais recomendada dos Protocol Buffers é a proto3, que apresenta uma sintaxe mais simples, novos recursos úteis e suporte a uma ampla variedade de linguagens, como Java, Python, Go, C++, JavaScript, entre outras. Embora o proto2 ainda esteja disponível, o proto3 é preferido no uso com gRPC para evitar problemas de compatibilidade e aproveitar o suporte ampliado a múltiplas linguagens. Você pode consultar as outras versões do protobuffs e a diferença entre elas na documentação oficial do Protocol Buffers.

Para entendermos ainda mais sobre as classes criadas através do compilador proto c vamos analisar esses dois trechos de código:

```
      syntax = "proto3";

     package veiculo;

     option  java_multiple_files = true;
     option java_package = "dev.ghlima.veiculo";
```

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

```
message Veiculo {
 int32 id = 1;
 string marca = 2;
 string modelo = 3;
 int32 ano = 4;
}
```

Código-fonte 3 – Exemplo de arquivo .proto
Fonte: elaborado pelo autor (2024)

O trecho acima exemplifica um arquivo proto para representar uma mensagem de veículo, onde o compilador proto c compila esse arquivo para uma classe java. Podemos utilizá-la como no exemplo abaixo:

```
package dev.ghlima;

import dev.ghlima.veiculo.Veiculo;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class Demo2 {
   private static final Logger log =
LoggerFactory.getLogger(Demo2.class);

   public static void main(String[] args) {



      //criar um veiculo
      var veiculo1 = criaVeiculo();

      //criar outro veiculo com uma nova instancia porem
mesmos valores
      var veivulo2 = criaVeiculo();

      //comparar os veiculos
      log.info("equals {}", veiculo1.equals(veivulo2));
      log.info("== {}", (veiculo1 == veivulo2));


      var veiculo3 =
veiculo1.toBuilder().setModelo("Gol").build();
      log.info("Veiculo 3: {}", veiculo3);


      //comparar os veiculos
      log.info("equals {}", veiculo1.equals(veiculo3));
      log.info("== {}", (veiculo1 == veiculo3));
```

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

```
            //null?
            var                    veiculo4                    =
veiculo3.toBuilder().clearAno().build();
            log.info("Veiculo 4: {}", veiculo4);

        }

        public static Veiculo criaVeiculo() {
            return Veiculo.newBuilder()
                    .setId(1)
                    .setAno(2021)
                    .setModelo("Fusca")
                    .setMarca("Volkswagen")
                    .build();
        }
    }
```

Código-fonte 4 – Exemplo de classe java para exemplificar imutabilidade em instâncias de classes compiladas pelo protoc
Fonte: elaborado pelo autor (2024)

Através desse exemplo podemos entender conceitos como imutabilidade, igualdade de objetos e limpeza de atributos.

Quando criamos uma instância de Veículo usando o método newBuilder(), estamos configurando os valores dos campos desejados, como id, ano, modelo e marca. Após chamar o método build(), essa instância se torna imutável, ou seja, não pode ser alterada diretamente. Se quisermos modificar algum campo, precisamos usar o método toBuilder(), que cria um novo builder baseado na instância original. Esse builder permite alterar os valores desejados antes de chamar novamente build() para criar uma nova instância. Essa característica garante que os objetos originais permaneçam intactos, evitando efeitos colaterais e assegurando a integridade dos dados. No exemplo, ao modificar o campo modelo de "Fusca" para "Gol", criamos veiculo3, uma nova instância derivada de veiculo1, mas sem alterar os valores originais de veiculo1.

A igualdade entre instâncias é verificada pelo método equals(), que compara os valores de todos os campos das mensagens para determinar se elas são semanticamente iguais. No caso de veiculo1 e veiculo2, embora sejam objetos diferentes na memória, o método equals() retorna true, pois todos os campos possuem os mesmos valores. Isso contrasta com o operador ==, que verifica apenas se as referências de memória são as mesmas e, portanto, retorna false. Por outro lado, ao

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

comparar veiculo1 com veiculo3, o método equals() retorna false, já que o campo modelo foi alterado em veiculo3.

Outro ponto interessante ocorre ao utilizar o método clear() no builder. No Proto3, esse método redefine o campo para o seu valor padrão, em vez de removê-lo ou deixá-lo como null. Por exemplo, ao chamar clearAno() no builder de veiculo3, o campo ano é redefinido para o valor padrão de um inteiro, que é 0. Isso é consistente com a filosofia do Proto3, onde todos os campos possuem valores padrão intrínsecos: números inteiros são 0, strings são vazias ("") e booleanos são false. Essa abordagem elimina a necessidade de lidar com valores nulos, tornando o comportamento das classes mais previsível e reduzindo o risco de erros em tempo de execução.

Nessa aula, vimos alguns tipos comuns do gRPC e aprendemos sobre composição, collection e enum, que são features que o gRPC nos fornece para trabalhar com modelos de dados complexos. Mas existem também alguns tipos fornecidos pelo próprio pacote do protobuf chamado de well-known types. São mensagens pré-definidas pelo Google no Protobuf que ajudam a resolver problemas comuns de modelagem de dados. Você pode usar esses tipos prontos em vez de criar do zero mensagens para coisas como:

- Valores opcionais de inteiros, booleanos ou strings (chamados de wrappers).
- Marcação de tempo, data/hora (Timestamp).
- Intervalos de tempo (Duration).

Eles economizam tempo, evitam erros e são otimizados para interoperabilidade. Para conhecer todos os Protocol Buffers Well-Known Types, você pode consultar a documentação oficial.

Com isso, conseguimos entender ainda mais como o protobuf funciona. Ele se baseia em arquivos .proto, que atuam como contratos para definir mensagens (os dados a serem transmitidos) e serviços (os métodos que manipulam esses dados). A simplicidade e eficiência desse protocolo se destaca no cenário de comunicação entre sistemas.

As mensagens são compostas por campos, cada um identificado por um nome, um tipo e, o mais importante, um número de campo (ou tag). Durante a transmissão, o Protobuf não envia os nomes dos campos; em vez disso, utiliza as tags numéricas

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

associadas, otimizando o tamanho da mensagem. Essa compactação é fundamental para sua eficiência. Por exemplo, um campo chamado "sobrenome" com a tag 1 e valor "Sam" seria serializado apenas como "1: Sam". Essa abordagem reduz significativamente o volume de dados transmitidos.

Os números de campo têm regras importantes que governam seu uso:

- As tags podem ser atribuídas em qualquer ordem, mas devem ser únicas dentro da mesma mensagem.
- Existe um intervalo reservado (19.000 a 19.999) que não pode ser utilizado.
- Para maximizar a eficiência, tags de 1 a 15 ocupam apenas 1 byte, enquanto tags de 16 a 2047 consomem 2 bytes. Por isso, é recomendável atribuir números menores a campos usados com frequência.

Outra prática fundamental ao projetar mensagens é a organização e modularidade. Mensagens menores e bem divididas são mais fáceis de gerenciar e permitem reuso em diferentes partes do sistema. Caso um campo precise ser removido no futuro, é uma boa prática marcar seu número como reservado no arquivo .proto, evitando conflito caso ele seja reutilizado inadvertidamente.

Durante o processo de serialização, apenas os campos definidos e com valores diferentes dos padrões (como false para booleanos ou 0 para inteiros) são transmitidos, economizando espaço. A deserialização, por sua vez, utiliza o mesmo arquivo .proto para mapear os números de campo recebidos de volta para os respectivos nomes e tipos. Essa simetria entre cliente e servidor garante que os dados sejam interpretados corretamente.

A partir do arquivo .proto, o Protobuf e o gRPC geram automaticamente o código necessário em várias linguagens, como Java, Python e C#. Isso permite criar objetos e serviços fortemente tipados, com métodos para construir, serializar e manipular mensagens.

A eficiência do Protobuf vai além da compactação de dados. Ao evitar redundâncias (como valores padrão) e priorizar números menores para os campos mais usados, ele garante alto desempenho mesmo em redes limitadas ou ambientes que exigem baixa latência. Além disso, sua interoperabilidade entre diferentes sistemas e linguagens o torna uma escolha robusta para arquiteturas modernas.

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

Por fim, ao modificar mensagens já em produção, é crucial seguir boas práticas de versionamento, como manter a compatibilidade retroativa sempre que possível. Isso significa evitar alterações que quebrem a interpretação de mensagens já existentes, como a reutilização indiscriminada de tags. Um design cuidadoso e colaborativo entre as equipes, aliado às práticas recomendadas do Protobuf, garante um sistema eficiente, flexível e sustentável a longo prazo.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

O QUE VOCÊ VIU NESTA AULA?

Nesta aula exploramos os conceitos fundamentais do RPC e sua implementação moderna com o gRPC, entendendo como ele facilita a comunicação eficiente entre sistemas distribuídos. Aprendemos que o gRPC utiliza o Protobuf como mecanismo padrão para serialização de dados, garantindo mensagens compactas e de alto desempenho.

Compreendemos como os arquivos .proto atuam como contratos, definindo mensagens e serviços, e como o compilador protoc gera classes fortemente tipadas em diversas linguagens. Vimos exemplos práticos de modelagem de mensagens, como no domínio de veículos, aplicando conceitos como imutabilidade, igualdade de objetos e redefinição de valores padrão.

Também exploramos os números de campo (tags), essenciais para a eficiência da serialização, e discutimos boas práticas de organização e versionamento, como o uso de tags menores para otimização e a marcação de campos como reservados. Por fim, analisamos os well-known types, como wrappers e timestamps, que simplificam a modelagem de dados complexos.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

REFERÊNCIAS

MICROSOFT. **Chamada de Procedimento Remoto (RPC)**. 2024. Disponível em: <https://learn.microsoft.com/pt-br/windows/win32/rpc/rpc-start-page>. Acesso em: 09 dez. 2024.

GRPC.io. **Well-Known Types**. 2024. Disponível em: <https://grpc.io/search/?q=well%20known%20types>. Acesso em: 09 dez. 2024.

IBM. **Chamada de Procedimento Remoto (RPC**). 2024. Disponível em: <https://www.ibm.com/docs/pt-br/aix/7.3?topic=concepts-remote-procedure-call>. Acesso em: 09 dez. 2024.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

PALAVRAS-CHAVE

Remote Procedure Call. gRPC. Protocol Buffers. Serialização. Deserialização. Mensagens Escaláveis. Interoperabilidade. Compactação de Dados. Microsserviços. HTTP/2. Streaming. Eficiência em Redes. Imutabilidade. Contratos Protobuf. Well-Known Types. Versionamento de Mensagens. Boas Práticas. Integração Cliente-Servidor. Cloud-Native. CNCF. Automação de Código.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com



PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com